

The Gaia API

Objectives

1. Enable a user for Gaia API access and verify authentication against the SMS.
2. Run Gaia API calls via `mgmt_cli` using the `--context gaia_api` switch.
3. Navigate the local Gaia API documentation portal and pick the correct API version.
4. Drive Gaia API calls through the Management API as a proxy — one session reaching every managed gateway.
5. Make a real configuration change via Gaia API, verify it in the portal, then revert.

Lab Environment

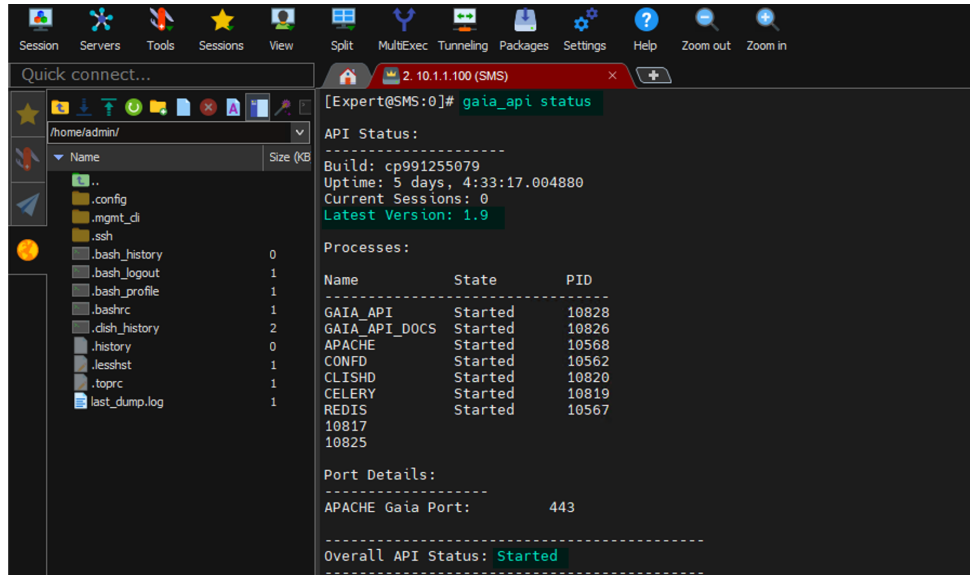
Resource	Address	Credentials
SMS	10.1.1.100	admin / Cpwins!1
GW	10.1.1.111	admin / Cpwins!1
Jump server (Windows)	via MobaXterm bookmark	—

Pre-Flight

The **Gaia API** manages the **OS layer** of Check Point appliances — hostname, interfaces, routes, DNS, NTP, users, snapshots — everything that lives in Gaia rather than in the security policy. It is a separate API from the Management API you used in Labs 1–3: every gateway and SMS runs its own Gaia API server, accessed via the same `mgmt_cli` tool but switched to the Gaia context.

1. Continue from your Lab 3 SSH session to **SMS** (10.1.1.100). Reconnect if closed.
2. Verify the Gaia API server is running on the SMS:

```
gaia_api status
```



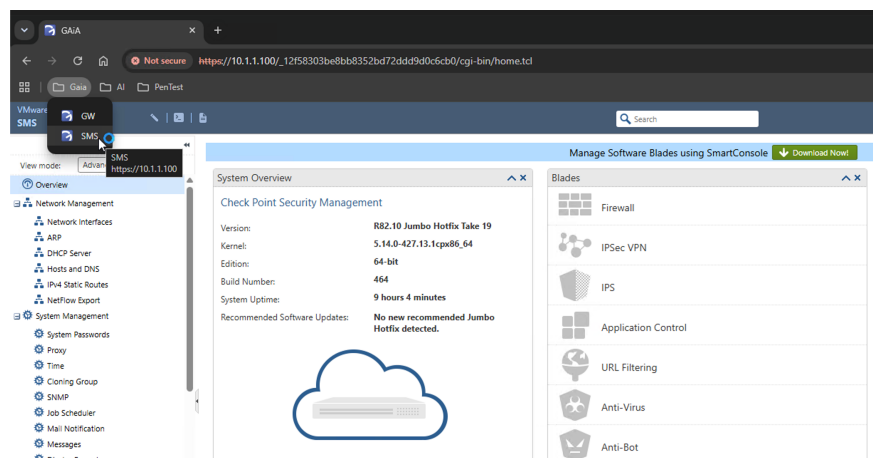
```
[Expert@SMS:0]# gaia_api status
API Status:
-----
Build: cp991255079
Uptime: 5 days, 4:33:17.004880
Current Sessions: 0
Latest Version: 1.9

Processes:
-----
Name           State      PID
-----
GAIA_API       Started    10828
GAIA_API_DOCS  Started    10826
APACHE         Started    10568
CONFd          Started    10562
CLISHd         Started    10820
CELERY         Started    10819
REDIS          Started    10567
10817
10825

Port Details:
-----
APACHE Gaia Port:      443

-----
Overall API Status: Started
```

- Open a browser on the jump server and confirm you can reach the Gaia Portal:
<https://10.1.1.100>, login admin / Cpwins!1.

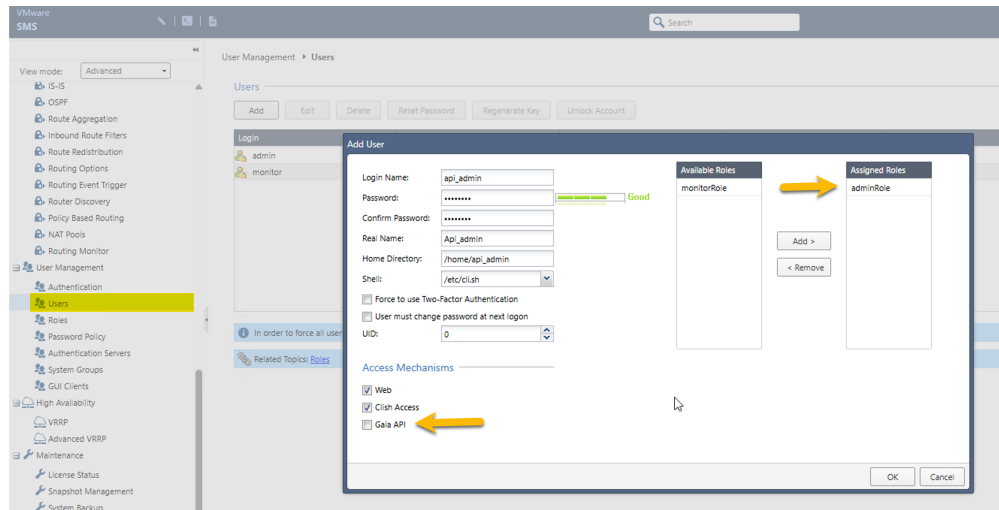


Task 1: Enable a User for Gaia API Access (15 Minutes)

Gaia API access is **not enabled by default** for new users. Only the predefined admin has it. In production, you want a dedicated, least-privilege account for automation — that is what we set up now.

1a — Create the user in the Gaia Portal

- In the Gaia Portal at <https://10.1.1.100>, navigate to **User Management** → **Users**.
- Add a new user: `api_admin`, password `Cpwins!1`, default shell `/etc/cli.sh`, `AdminRole`. Leave Gaia API access unchecked for now.



1b — Try to authenticate — expect failure

From the SMS SSH session, try logging in as the new user:

```
mgmt_cli -u api_admin -p 'Cpwins!1' --context gaia_api show hostname
```

You will get an authentication error. The user exists but does not yet have Gaia API permission. Note the password is in **single quotes** to prevent bash from history-expanding the !1 (same trap as Lab 1).

```
[Expert@SMS:0]# mgmt_cli -u api_admin -p 'Cpwins!1' --context gaia_api show hostname
code: "err_login_failed_wrong_username_or_password"
errors: "Login authentication failed"
message: "Login Failed Due to Wrong Username or Password" ←
```

1c — Grant the permission

```
gaia_api access -u api_admin -e true
```

The command returns nothing on success. In the Gaia Portal → User Management → api_admin, the Gaia API checkbox is now checked.

Equivalent via clish: add rba user api_admin access-mechanisms Gaia-API

1d — Re-run and confirm success

Run the same command from 1b. This time it succeeds and returns the SMS hostname.

```
[Expert@SMS:0]# gaia_api access -u api_admin -e true
[Expert@SMS:0]# mgmt_cli -u api_admin -p 'Cpwins!1' --context gaia_api show hostname
name: SMS
```

Task 2: Drive Gaia API via mgmt_cli (15 Minutes)

The same `mgmt_cli` tool you used in Lab 1, but with `--context gaia_api` — the same session-file pattern, the same JSON output discipline.

2a — A read call (no login required when using `-u / -p` inline)

```
mgmt_cli -u api_admin -p 'Cpwins!1' --context gaia_api show-routes -f json |  
jq '.objects | length'
```

Returns the number of routes the SMS knows about. JSON and jq from Lab 2 apply identically here.

```
[Expert@SMS:0]# mgmt_cli -u api_admin -p 'Cpwins!1' --context gaia_api show-routes -f json | jq '.objects | length'  
4  
[Expert@SMS:0]#
```

2b — The session-file pattern (same as Lab 1)

```
# 1. Open a Gaia API session  
mgmt_cli login -u api_admin -p 'Cpwins!1' --context gaia_api -f json >  
/tmp/gaia.txt  
  
# 2. Use the session for multiple calls (context is inherited from the session  
file)  
mgmt_cli show hostname -s /tmp/gaia.txt -f json  
mgmt_cli show version -s /tmp/gaia.txt -f json | jq -r '["product-version"]'  
mgmt_cli show dns -s /tmp/gaia.txt -f json  
  
# 3. Close the session  
mgmt_cli logout -s /tmp/gaia.txt
```

Key point: the `--context gaia_api` flag was stored inside `/tmp/gaia.txt` at login. Subsequent calls do not need to repeat it. Compare this with Lab 1's `/tmp/sid.txt` — same mechanic, different context. You can have both session files open and switch between the Management and Gaia surfaces by changing which one you point `-s` at.

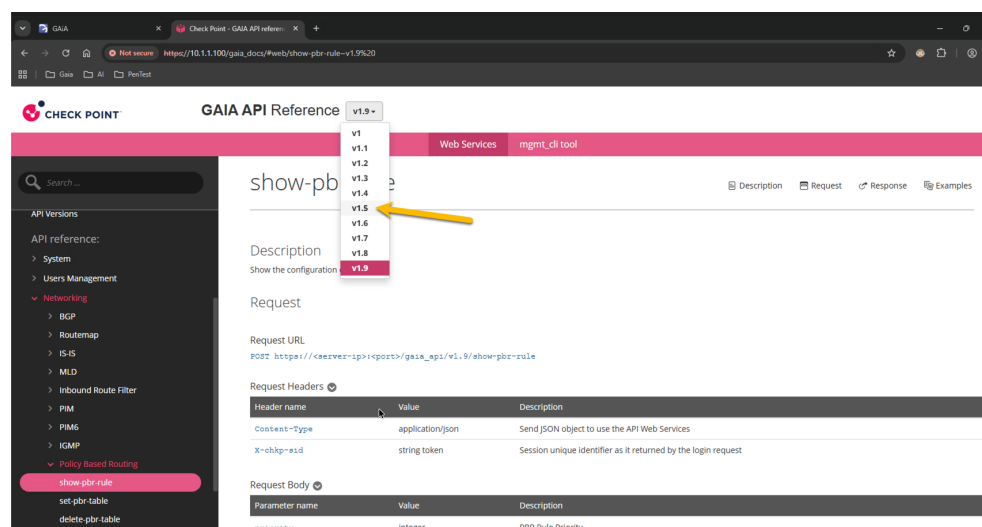
Task 3: The Gaia API Documentation Portal (10 Minutes)

Every Gaia API server ships its own documentation portal. Public copy lives at `sc1.checkpoint.com`; the same docs are also served locally by every appliance.

8. Browser: `https://10.1.1.100/gaia_docs/` (the trailing slash is required).

9. Navigate Networking → Routes → show-routes. The page shows two tabs: Web Services (raw HTTP) and mgmt_cli. Switch between them and note the syntax differences.
10. Search for policy-based-routing. Then switch the version drop-down from v1.9 to v1.5 — the results disappear. PBR was added in v1.7. You can pin a request to a specific version with --version.

Lesson. When a customer asks "can I do X via the Gaia API?", the answer is: check the docs portal for their version. New commands ship per Gaia API version; what works on R82.10 may not work on an older lab. Changelog lives in sk143612.



Task 4: Gaia API Through the Management Proxy (15 Minutes)

This is the pattern customers will pay you to know. Up to this task you targeted the SMS's local Gaia API directly. Hitting Gaia on **every managed gateway** directly means N login sessions and N SSH paths. The Management API exposes a **proxy mode** — one Mgmt session reaches Gaia on any gateway it manages.

4a — The wrong credentials (do not run; here to read)

```
# Anti-pattern – the api_admin user only exists in the Gaia OS, not in the
Mgmt API.
mgmt_cli -u api_admin -p 'Cpwins!1' gaia-api/show-hostname target 10.1.1.111 -
f json
```

Authentication fails because you are now talking to the **Management API**, which has its own user store. `api_admin` only exists in Gaia.

4b — The right call

```
mgmt_cli -u admin -p 'Cpwins!1' gaia-api/show-hostname target 10.1.1.111 -f json
```

Three things to notice about this command:

1. The credentials are admin — the **Management API** admin, not the Gaia API user.
2. No `--context gaia_api` flag. Instead the verb is written as `gaia-api/<command>` — the slash tells the Management API "forward this to Gaia on the target."
3. The target `10.1.1.111` parameter says which managed gateway to relay to. You can iterate this across your whole fleet.

```
[Expert@SMS:0]# mgmt_cli -u api_admin -p 'Cpwins!1' gaia-api/show-hostname target 10.1.1.111 -f json
code: "err_login_failed"
message: "Authentication to server failed."

[Expert@SMS:0]# mgmt_cli -u admin -p 'Cpwins!1' gaia-api/show-hostname target 10.1.1.111 -f json
{
  "command-name" : "show-hostname",
  "response-message" : {
    "name" : "GW"
  }
}
```

Why this matters. Customers with dozens or hundreds of gateways do not want to maintain Gaia API accounts on each device. The proxy pattern lets a single Management API user reach Gaia everywhere — auditable in one place, with the Management server doing the credential management.

Task 5: Make a Real Change, Verify, and Revert (10 Minutes)

Add a comment to the GW's eth0 interface via Gaia API on the gateway, verify it in the Gaia Portal, then revert.

5a — Read the current state

```
mgmt_cli -u admin -p 'Cpwins!1' gaia-api/show-physical-interface name "eth0"
target 10.1.1.111 -f json | jq '{name, ipv4_address: .["ipv4-address"],
comments}'
```

Note the current comments field (empty). The target is the GW at 10.1.1.111.

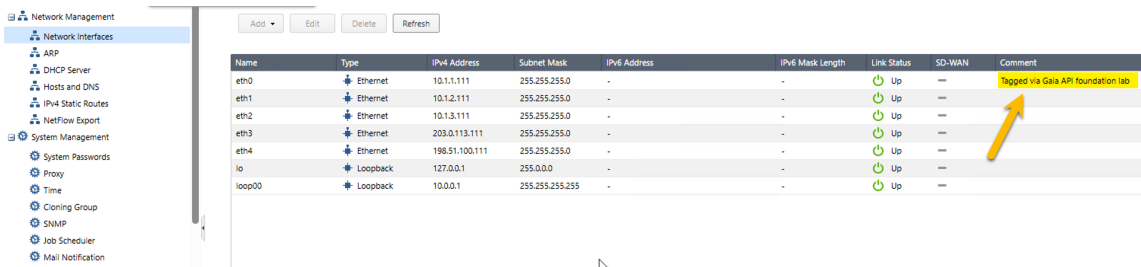
```
[Expert@SMS:0]# mgmt_cli -u admin -p 'Cpwins!1' gaia-api/show-physical-interface name "eth0" target 10.1.1.111 -f json | jq '{name, ipv4_address: .["ipv4-address"], comments}'
{
  "name": null,
  "ipv4_address": null,
  "comments": null
}
```

5b — Set a comment

```
mgmt_cli -u admin -p 'Cpwins!1' gaia-api/set-physical-interface name "eth0"
comments "Tagged via Gaia API foundation lab" target 10.1.1.111 -f json
```

5c — Verify in the Gaia Portal

Browser: <https://10.1.1.111> → Network Management → Network Interfaces → eth0 → comments. The new text is there.



Name	Type	IPv4 Address	Subnet Mask	IPv6 Address	IPv6 Mask Length	Link Status	SD-WAN	Comment
eth0	Ethernet	10.1.1.111	255.255.255.0	-	-	Up	—	Tagged via Gaia API foundation lab
eth1	Ethernet	10.1.2.111	255.255.255.0	-	-	Up	—	
eth2	Ethernet	10.1.3.111	255.255.255.0	-	-	Up	—	
eth3	Ethernet	203.0.113.111	255.255.255.0	-	-	Up	—	
eth4	Ethernet	196.51.100.111	255.255.255.0	-	-	Up	—	
lo	Loopback	127.0.0.1	255.0.0.0	-	-	Up	—	
loop00	Loopback	10.0.0.1	255.255.255.255	-	-	Up	—	

5d — Revert

```
mgmt_cli -u admin -p 'Cpwins!1' gaia-api/set-physical-interface name "eth0"
comments "" target 10.1.1.111 -f json
```

Checkpoint (5 Minutes)

Write two short answers (one sentence each):

4. **What is the difference between the Gaia API and the Management API?**
5. **Customer scenario:** "We need a nightly inventory of every gateway: hostname, OS version, all interfaces, all routes. The CMDB will consume the JSON." Which API and which calling pattern do you recommend, and why?

Expected direction for #2: Management API as the proxy entry point, gaia-api/show-hostname, show-version, show-physical-interfaces, show-routes, iterated across the list of target gateways (from show-gateways-and-servers). One Mgmt session, no per-gateway accounts to maintain, all output in JSON ready for the CMDB.

References

6. **Gaia API official reference (R82.10 latest):**
sc1.checkpoint.com/documents/latest/GaiaAPIs/
7. **Local Gaia API docs:** https://<appliance-ip>/gaia_docs/

8. **Gaia API changelog (per-version):** `sk143612`
9. **Deep-dive workshop (full 2-hour version with Python SDK + Postman):**
Workshops/Gaia-API-Workshop/ and the source repo at
`github.com/alshawwaf/Gaia_API_Workshop`
10. **Check Point Python SDK** (used in the workshop, optional for Foundation):
`github.com/CheckPointSW/cp_mgmt_api_python_sdk`